

EurekaVault Comprehensive Reference

Evidence-backed implementation and roadmap snapshot

Repository: `kirolossedra/Prompt-management`

Snapshot: 7e45248 – 20 August 2026 (GitHub UTC)

Contents

1 Purpose and Evidence Standard	2
2 Current Product Shape	2
3 Runtime Architecture	2
4 Data and Version-Control Model	3
5 Knowledge and Methodology Features	3
5.1 Prompt files	3
5.2 Prompt lineage	3
5.3 Mindsets and Preferences	3
6 AI Boundary	3
7 Prompt Blocks	4
8 EurekaVault Identity	4
9 Spring Boot Migration	4
10 CI/CD	4
11 Security Notes	5
12 Commit-Backed Delivery History	5
13 Actual GitHub Epics	5

14 Open Issue Ledger	6
15 Removed Unsupported Roadmap Commitments	6
16 Roadmap Maintenance Rule	6

1 Purpose and Evidence Standard

This reference describes the implementation present at repository commit `7e45248a408e372088d18cab74faef5a79523`. Current source code and Git commits are authoritative for what exists. GitHub Issues, Issue comments, and actual labels are authoritative for explicitly raised open scope.

The roadmap intentionally does not infer future delivery dates, priorities, completion percentages, dependencies, or strategic initiatives. An open Issue is also not treated as proof that its implementation is absent: several Issues remain open after corresponding code landed.

2 Current Product Shape

EurekaVault is a private, version-controlled Prompt and methodology workspace. Its current implementation combines:

- a React 19 + TypeScript + Vite frontend;
- Firebase Authentication and owner-scoped Realtime Database storage;
- automatic Prompt-local version history and Git-style comparison;
- Global Version snapshots;
- Prompt attachments and directed Prompt lineage;
- Mindsets, Preferences, Decisions, activity, and Achievements;
- conventional Prompt/history search;
- authenticated Gemini-backed Finder, Repurposer, Mixer, and Prompt Blocks;
- a Spring Boot 4.1 / Java 21 backend foundation hosted through Render/Docker;
- GitHub Actions quality and deployment gates.

3 Runtime Architecture

```
React browser application
|-- Firebase Authentication
|-- Firebase Realtime Database
|   current owner-scoped vault CRUD path
|
|-- Netlify Functions
|   current authenticated Gemini boundary
|   |-- Semantic Finder
|   |-- Repurposer
|   |-- Mixer
|   '-- Prompt Blocks transformation
|
'-- Spring Boot / Render
    incremental 3-tier migration foundation
    current public application role: infrastructure health
```

The physical deployment is multi-service, but the application data path is not yet fully 3-tier. The browser still performs direct Firebase data access and AI requests still use Netlify Functions. The Spring Boot service must therefore be described as a migration foundation, not as the completed application API.

4 Data and Version-Control Model

The direct hierarchy is:

Endeavor -> Task -> Prompt -> Prompt Version

The owner-private data root is `intellectVault/users/{firebaseUid}/`. Firebase rules require the authenticated UID to match the path UID.

Prompt history is automatic: creating a Prompt establishes Version 1 and saving a changed Prompt creates a new historical snapshot. Restoring historical content creates another current version rather than deleting later history. Global Versions are separate, deliberate vault-wide snapshots.

5 Knowledge and Methodology Features

5.1 Prompt files

Prompt attachments are Base64-backed and bounded by application limits of 2 MiB per file, 20 files per Prompt, and 10 MiB aggregate per Prompt.

5.2 Prompt lineage

Directed `inspired-by` / `inspires` relationships are explicit user-created lineage edges. Duplicate and cyclic edges are rejected. GitHub Issue #15 asks whether an “instance” model should replace this concept, but that Issue is a design proposal, not an approved migration.

5.3 Mindsets and Preferences

Mindsets are scoped methodology artifacts. Mindset Construction deterministically assembles selected Prompt material and does not call Gemini. Preferences are persisted according to the scopes implemented in the current domain model.

6 AI Boundary

Current AI calls go through authenticated Netlify Functions; the Gemini key is not sent to the browser. The client supplies a Firebase ID token and the server boundary verifies the authenticated user context before invoking Gemini.

Data is scoped per operation. Finder receives a bounded searchable Prompt corpus; Repurposer receives one source Prompt plus an objective; Mixer receives selected Prompt sources; a Prompt Blocks transform receives only the content and ordered constraints required by that block.

Generated content is non-destructive until an explicit persistence action occurs.

7 Prompt Blocks

Prompt Blocks is a deterministic Prompt-processing directed acyclic graph rather than a generic autonomous-agent framework. The MVP landed in [b556e64](#), with correctness fixes in [e079616](#) and [90b6799](#) and live-workspace refinements in [7e45248](#).

Implemented behavior includes:

- current or pinned Prompt inputs and direct user inputs;
- typed content and constraint connections;
- multiple Prompt transformation operations;
- Mindset and extracted-style constraints with explicit priority;
- As Is, Summarized, and Conclusion Only outputs;
- deterministic DAG validation, cycle rejection, and topological execution;
- inspectable intermediate results and downstream blocking on failures;
- temporary and persisted pipelines;
- editable Firebase-backed transformation prompts;
- explicit output persistence through the existing Prompt/version lifecycle;
- desktop visual editing and a dedicated mobile interaction model;
- deterministic beautification, printing, compact IDE-style controls, an output-intent warning, and reachable scrolling run output.

Issue #7 remains open and contains broader design exploration. This reference does not interpret every idea in that Issue as promised follow-up work.

8 EurekaVault Identity

The naming and visual identity work is supported by GitHub Issue #9 and commits including [3b61d70](#) and [349b24f](#). The code contains the EurekaVault name, epsilon mark, Alexandrian lighthouse motif, Greek identity copy, and a light-blue/gold/royal-red palette. The Issue remains open, so the rebrand is described as partially implemented rather than declared complete.

9 Spring Boot Migration

GitHub Issue #12 carries the **architecture** and **epic** labels. Its owner comment explicitly states: “It’s gonna be springboot.” Commit [4a3cc23](#) introduced the Spring Boot 4.1 / Java 21 service, health endpoint, Docker packaging, and Render configuration.

The migration is incremental. Current Firebase CRUD and current Netlify Gemini paths remain active. A deployed health endpoint does not by itself make the application functionally 3-tier.

10 CI/CD

Commit [3d2bfa9](#) introduced frontend and backend quality gates plus a post-CI production deployment workflow. The checks include frontend lint/tests/ build, Netlify Function syntax validation, Maven verification, and a Dockerized Spring Boot health smoke test. Commit [54373c9](#) added Netlify build-hook gating; later commits corrected CI issues.

GitHub Issue #16 remains open because the production workflow is not currently path-split: a frontend-only change can still cause the backend deployment job to run after successful main-branch CI.

11 Security Notes

- Realtime Database authorization is owner-scoped by Firebase UID.
- Gemini credentials stay on server-side execution boundaries.
- Prompt Blocks runtime output does not persist automatically.
- Prompt Blocks default transform strings exist in the client bundle and therefore are not secrets.

Commit 54373c9 accidentally tracked deployment-hook URLs in a local environment file. Commit daa6f7b removed that file, but Git history retains deleted values. Those hooks should be treated as exposed and rotated if that has not already happened. Their values are intentionally omitted here.

12 Commit-Backed Delivery History

Date	Commit-backed delivery evidence
2026-08-06	React conversion and core CRUD foundation (c80d143, 18ab918).
2026-08-07–08	Prompt copy/versioning, UI overhaul, Git-style diff, files, relationships, Achievements, Finder, Repurposer, and Mixer.
2026-08-18	Finder feedback learning, documentation upgrade, roadmap tooling, EurekaVault rebrand/identity, and navigation work.
2026-08-19	Spring Boot migration foundation and GitHub Actions CI/CD gate.
2026-08-20 UTC	Prompt Blocks MVP, correctness fixes, and Issues #17/#18 workspace refinements.

13 Actual GitHub Epics

At the inspected snapshot, exactly five Issues carry the GitHub **epic** label:

Issue	Epic	Implementation evidence state
#3	Configurable base prompts + revision history	Partial only: Prompt Blocks transform prompts are editable; broad epic remains open.
#4	Architecture helper AI	No implementation evidence found.
#7	Prompt Blocks	MVP implemented; Issue remains open.
#11	CI/CD	Core quality/deployment gate implemented; Issue remains open.
#12	2-tier to 3-tier	Spring Boot foundation implemented; functional migration remains in progress.

No Issue has a GitHub Milestone assigned in the inspected snapshot. This reference therefore does not invent milestone dates or sequencing.

14 Open Issue Ledger

All Issues #1–#18 are open in GitHub. Important distinctions include:

- #2, #4, #5, #6, and #13 have no implementation evidence found;
- #3 is partial because Prompt Blocks implements only a feature-specific editable transformation-prompt mechanism;
- #7 and #11 have major implementation while remaining open;
- #8 and #14 remain unresolved frontend bugs;
- #9 and #10 are partially evidenced but remain open;
- #12 is explicitly in progress;
- #15 is a design proposal rather than an approved model change;
- #16 records the current deployment-coupling question;
- #17 and #18 were directly addressed by 7e45248 but remain open.

15 Removed Unsupported Roadmap Commitments

Older generated documentation presented markup-defined hierarchy parsing, collaboration, formal Mindset inheritance/conflict semantics, Preference precedence, and workspace deletion as future product initiatives. The inspected GitHub Issue set contains no corresponding Issue/epic for those items. They are therefore not part of the evidence-backed roadmap.

Current facts may still be stated—for example, the workspace is owner-private— without turning the absence of another feature into a promise to build it.

16 Roadmap Maintenance Rule

For future updates:

1. use current code and commits to establish delivered behavior;
2. use GitHub Issues/comments/labels to establish explicitly raised scope;
3. keep GitHub open/closed state separate from implementation evidence;
4. use actual GitHub Milestones only if the owner creates them;
5. do not infer delivery dates, percentages, priorities, dependencies, or epics.

This preserves a roadmap that reports the owner’s project rather than silently turning analysis into commitments.